

EXPERIMENT-7: Servlet controller for web interaction.

Design a controller with servlet that provides interaction with the application developed in experiment 1 and the database created in experiment 5.

Objective : Connect the frontend web app to backend servlet logic and database operations.

Tools : Java Servlets, JDBC, HTML forms, backend routing

Outcome : A servlet-based web controller integrating the shopping app with database CRUD.

Answer:

Handle form requests in servlet methods, use JDBC to update the database, and return the response to the client.

📁 File Structure

BookstoreServletApp/

- ├─ src/com/lab/bookstore/
 - | └─ Book.java -> POJO model
 - | └─ DBUtil.java -> JDBC connection helper
 - | └─ BookServlet.java -> Controller: routes GET/POST, runs SQL, renders HTML
- ├─ WebContent/
 - | └─ index.html -> Landing page
- ├─ WEB-INF/
 - | └─ web.xml -> Deployment descriptor
- ├─ lib/ -> put mysql-connector-j-x.x.x.jar here
- └─ sql/
 - └─ bookstore.sql -> creates database + books table + sample rows

File Name: Book.java

Code:

```
package com.lab.bookstore;
```

```
public class Book {
```

```
private int id;  
private String title;  
private String author;  
private double price;
```

```
public Book() {  
}
```

```
public Book(int id, String title, String author, double price) {  
    this.id = id;  
    this.title = title;  
    this.author = author;  
    this.price = price;  
}
```

```
public int getId() {  
    return id;  
}
```

```
public void setId(int id) {  
    this.id = id;  
}
```

```
public String getTitle() {  
    return title;  
}
```

```
public void setTitle(String title) {  
    this.title = title;  
}
```

```
public String getAuthor() {  
    return author;  
}
```

```

    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }
}

```

File Name: BookServlet.java

```
package com.lab.bookstore;
```

```

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

```

```

@WebServlet("/books")
public class BookServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    // ----- GET: view list / edit form / delete -----
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String action = request.getParameter("action");

        try {
            if ("delete".equals(action)) {
                int id = Integer.parseInt(request.getParameter("id"));
                deleteBook(id);
                response.sendRedirect("books"); // PRG pattern: redirect after action
                return;
            }

            if ("edit".equals(action)) {
                int id = Integer.parseInt(request.getParameter("id"));
                Book book = getBookById(id);
                renderPage(response, listAllBooks(), book, null);
                return;
            }

            // default: list view
            renderPage(response, listAllBooks(), null, null);

        } catch (SQLException e) {
            renderPage(response, new ArrayList<>(), null, "Database error: " + e.getMessage());
        }
    }
}

```

```

    }
}

// ----- POST: add / update -----
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String action = request.getParameter("action");
    String title = request.getParameter("title");
    String author = request.getParameter("author");
    String priceStr = request.getParameter("price");

    try {
        double price = Double.parseDouble(priceStr);

        if ("update".equals(action)) {
            int id = Integer.parseInt(request.getParameter("id"));
            updateBook(new Book(id, title, author, price));
        } else {
            // default action = add
            insertBook(new Book(0, title, author, price));
        }

        response.sendRedirect("books"); // back to list after write

    } catch (NumberFormatException e) {
        renderPage(response, listAllBooks(), null, "Price must be a valid number.");
    } catch (SQLException e) {
        renderPage(response, new ArrayList<>(), null, "Database error: " + e.getMessage());
    }
}

```

```

private List<Book> listAllBooks() throws SQLException {
    String sql = "SELECT id, title, author, price FROM books ORDER BY id";
    List<Book> books = new ArrayList<>();

    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            books.add(new Book(
                rs.getInt("id"),
                rs.getString("title"),
                rs.getString("author"),
                rs.getDouble("price")));
        }
    }

    return books;
}

```

```

private Book getBookById(int id) throws SQLException {
    String sql = "SELECT id, title, author, price FROM books WHERE id = ?";

    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setInt(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return new Book(
                    rs.getInt("id"),
                    rs.getString("title"),
                    rs.getString("author"),

```

```

        rs.getDouble("price"));
    }
}
return null;
}

```

```

private void insertBook(Book book) throws SQLException {
    String sql = "INSERT INTO books (title, author, price) VALUES (?, ?, ?)";

    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, book.getTitle());
        ps.setString(2, book.getAuthor());
        ps.setDouble(3, book.getPrice());
        ps.executeUpdate();
    }
}

```

```

private void updateBook(Book book) throws SQLException {
    String sql = "UPDATE books SET title = ?, author = ?, price = ? WHERE id = ?";

    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, book.getTitle());
        ps.setString(2, book.getAuthor());
        ps.setDouble(3, book.getPrice());
        ps.setInt(4, book.getId());
        ps.executeUpdate();
    }
}

```

```

private void deleteBook(int id) throws SQLException {
    String sql = "DELETE FROM books WHERE id = ?";

    try (Connection conn = DBUtil.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setInt(1, id);
        ps.executeUpdate();
    }
}

```

```

private void renderPage(HttpServletResponse response, List<Book> books,
    Book editingBook, String errorMessage) throws IOException {

    response.setContentType("text/html;charset=UTF-8");
    try (PrintWriter out = response.getWriter()) {

        out.println("<!DOCTYPE html>");
        out.println("<html><head><title>Bookstore Admin</title>");
        out.println("<style>");
        out.println("body{font-family:Arial,sans-serif;max-width:800px;margin:30px");
        out.println("auto;padding:0 15px;}");
        out.println("table{width:100%;border-collapse:collapse;margin-top:15px;}");
        out.println("th,td{border:1px solid #ccc;padding:8px;text-align:left;}");
        out.println("th{background:#f0f0f0;}");
        out.println("form.inline{display:inline;}");
        out.println("fieldset{margin-bottom:20px;padding:15px;}");
        out.println(".error{color:#b00020;font-weight:bold;}");
        out.println("input[type=text],input[type=number]{padding:4px;margin:4px");
        out.println("0;width:95%;}");
        out.println("button{padding:6px 14px;margin-top:6px;cursor:pointer;}");
    }
}

```

```

out.println("</style></head><body>");

out.println("<h1>Bookstore Servlet Controller</h1>");
out.println("<p>Front end (Experiment 1) & rarr; Servlet controller & rarr; JDBC & rarr;
MySQL (Experiment 5)</p>");

if (errorMessage != null) {
    out.println("<p class='error'>" + escape(errorMessage) + "</p>");
}

// ---- Add / Edit form ----
boolean isEdit = editingBook != null;
out.println("<fieldset>");
out.println("<legend>" + (isEdit ? "Edit Book" : "Add New Book") + "</legend>");
out.println("<form method='post' action='books'>");
if (isEdit) {
    out.println("<input type='hidden' name='action' value='update'>");
    out.println("<input type='hidden' name='id' value='" + editingBook.getId() + "'>");
} else {
    out.println("<input type='hidden' name='action' value='add'>");
}
out.println("<label>Title</label><br>");
out.println("<input type='text' name='title' required value='"
    + (isEdit ? escape(editingBook.getTitle()) : "") + "'><br>");
out.println("<label>Author</label><br>");
out.println("<input type='text' name='author' required value='"
    + (isEdit ? escape(editingBook.getAuthor()) : "") + "'><br>");
out.println("<label>Price</label><br>");
out.println("<input type='number' step='0.01' name='price' required value='"
    + (isEdit ? editingBook.getPrice() : "") + "'><br>");
out.println("<button type='submit'>" + (isEdit ? "Update Book" : "Add Book") +
"</button>");
if (isEdit) {
    out.println("<a href='books'>Cancel</a>");
}

```

```

    }
    out.println("</form>");
    out.println("</fieldset>");

    // ---- Book table ----
    out.println("<table>");
    out.println("<tr><th>ID</th><th>Title</th><th>Author</th><th>Price</th><th>Actions</th></tr>");

    for (Book b : books) {
        out.println("<tr>");
        out.println("<td>" + b.getId() + "</td>");
        out.println("<td>" + escape(b.getTitle()) + "</td>");
        out.println("<td>" + escape(b.getAuthor()) + "</td>");
        out.println("<td>" + String.format("%.2f", b.getPrice()) + "</td>");
        out.println("<td>");
        out.println("<a href='books?action=edit&id=" + b.getId() + "'>Edit</a> | ");
        out.println("<a href='books?action=delete&id=" + b.getId()
            + "' onclick=\"return confirm('Delete this book?');\">Delete</a>");
        out.println("</td>");
        out.println("</tr>");
    }
    out.println("</table>");

    out.println("</body></html>");
}

private String escape(String input) {
    if (input == null) return "";
    return input.replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace(""", "&quot;");
}

```

```
        .replace("", "&#39;");
    }
}
```

File Name: bookstore.sql{

```
-- bookstore.sql
-- Run this in MySQL before deploying the servlet app.
```

```
CREATE DATABASE IF NOT EXISTS bookstore_db;
USE bookstore_db;
```

```
CREATE TABLE IF NOT EXISTS books (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    author VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL
);

-- Sample data matching Experiment 1's XML records
INSERT INTO books (title, author, price) VALUES
    ('Node.js Essentials', 'Sarah Mitchell', 299.00),
    ('Mastering XML Schemas', 'Rajesh Kumar', 450.50),
    ('Web Development with Express', 'Emily Chen', 375.00);
```

File Name: DBUtil.java

```
package com.lab.bookstore;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
```

```

* JDBC connection helper for the bookstore_db database
* created in Experiment 5.
*
* Update DB_USER / DB_PASSWORD to match your local MySQL setup.
*/
public class DBUtil {

    private static final String DB_URL =
        "jdbc:mysql://localhost:3306/bookstore_db?useSSL=false&serverTimezone=UTC";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "root"; // change to your MySQL password

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            throw new RuntimeException("MySQL JDBC Driver not found on classpath.", e);
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
    }
}

```

File Name : index.html

```

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Bookstore App</title>
    <style>

```

```
body { font-family: Arial, sans-serif; max-width: 600px; margin: 60px auto; text-align: center; }
a.button {
    display: inline-block; margin-top: 20px; padding: 10px 20px;
    background: #2c6fbb; color: #fff; text-decoration: none; border-radius: 4px;
}
</style>
</head>
<body>
    <h1>Welcome to the Bookstore</h1>
    <p>Front-end entry page (Experiment 1). Click below to manage books,
        served by the servlet controller talking to the MySQL database
        from Experiment 5.</p>
    <a class="button" href="books">Go to Bookstore Management</a>
</body>
</html>
```